Практика №2. Аудит готового клиент-серверного приложения

1. Ответы на вопросы

1. Ответ

Это протокол **TCP**. Использование константы `socket.SOCK_STREAM` указывает на потоковый сокет с гарантированной доставкой. Вызов метода `connect()` подтверждает установку постоянного соединения, что является обязательным атрибутом TCP-протокола.

2. Ответ

Это протокол **UDP**. Константа `socket.SOCK_DGRAM` обозначает работу с датаграммами без предварительного установления соединения. Метод `sendto()` отправляет данные напрямую на указанный адрес и порт, что характерно для UDP.

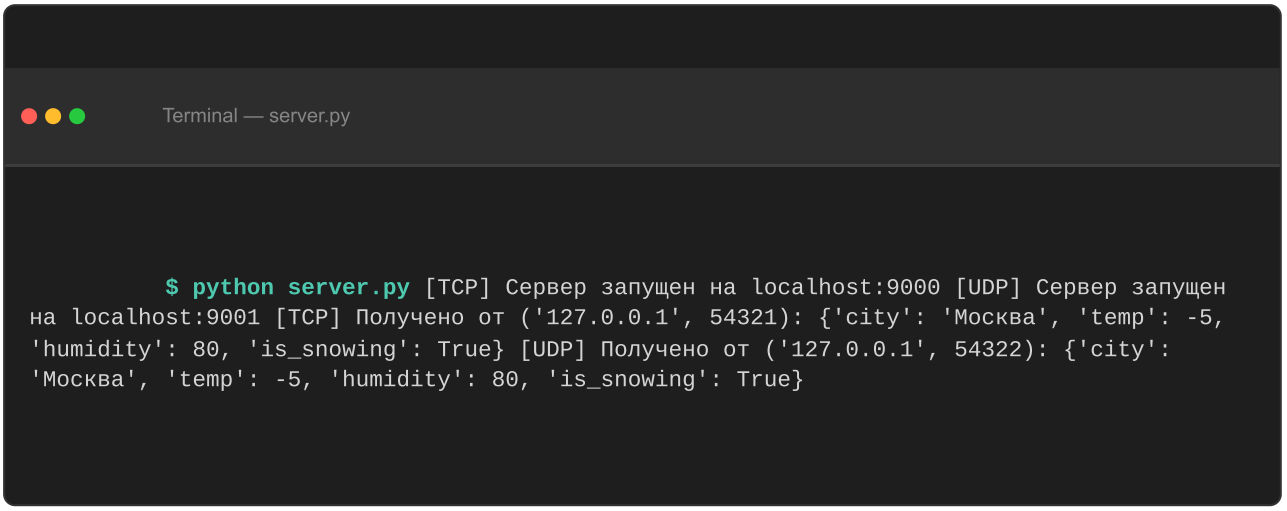
3. Ответ

Данные передаются в формате **JSON**, закодированном в байтовую строку UTF-8. Подтверждающая строка из `client.py`:
`client.send(json.dumps(message, ensure_ascii=False).encode("utf-8"))`

4. Ответ

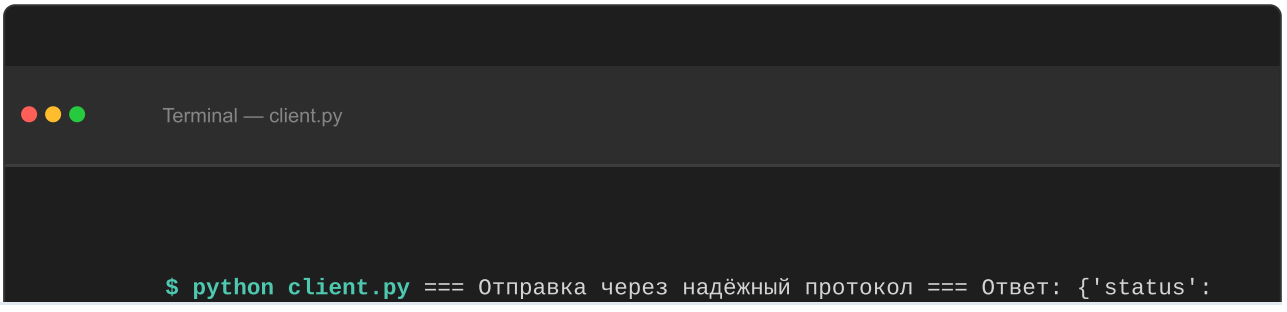
Сервер попытается декодировать строку вызовом `json.loads()`, что вызовет исключение `json.JSONDecodeError`. Перехватив ошибку, сервер выведет сообщение об ошибке в консоль и отправит клиенту JSON-ответ: `{"status": "ERROR", "reason": "invalid JSON"}`, после чего продолжит работу.

2. Скриншот окна сервера + описание



Описание: Сервер успешно запустил два независимых потока для прослушивания TCP и UDP портов. При поступлении запросов от клиента сервер корректно распарсил JSON и вывел полученную структуру данных в консоль.

Скриншот окна клиента + описание



3. Тестирование

1. Случай 1

Результат: Возникает ошибка `ConnectionRefusedError: [Errno 111] Connection refused`.
Причина: Протокол TCP требует обязательного установления соединения ("рукопожатия"). Поскольку сервер выключен и порт 9000 не прослушивается, операционная система отклоняет попытку подключения.

2. Случай 2

Результат: Ошибка подключения не возникает, но клиент ждёт 2 секунды и возвращает: `{'status': 'TIMEOUT'}`.
Причина: UDP отправляет датаграммы без установки соединения. Клиент не знает, доступен ли сервер, и ждет ответ до истечения таймаута (`client.settimeout(2.0)`), после чего срабатывает исключение `socket.timeout`.

3. Случай 3

Результат: В консоли сервера выводится `[TCP] Ошибка: не удалось разобрать JSON`, а клиент получает ответ `{'status': 'ERROR', 'reason': 'invalid JSON'}`.
Причина: Сервер перехватил исключение `json.JSONDecodeError` в блоке `try-except` и корректно обработал неформатные данные, не вызвав сбоя работы приложения.

4. Случай 4

Результат выполнения format_check.py:

Формат	Сгенерированная строка	Размер
JSON	`{"city": "Москва", "temp": -5, "humidity": 80, "is_snowing": true}`	67 байт
XML	`<weather><city>Москва</city><temp>-5</temp>...</weather>`	118 байт

Формат **JSON является более компактным** (67 байт против 118 байт), так как не требует дублирования избыточных открывающих и закрывающих тегов XML.

4. Вывод

Для данного приложения передачи метеоданных оптимальным выбором является связка **TCP + JSON** (или **UDP + JSON** в зависимости от требований к частоте обновлений):

- **Формат JSON:** является однозначным фаворитом за счет компактности (почти в 2 раза меньше байт по сравнению с XML), что экономит сетевой трафик, и простоты парсинга в Python.
- **Протокол передачи:**
  - **TCP** подходит, если важна гарантированная доставка каждого отчета о погоде без потерь.
  - **UDP** более предпочтителен, если датчики отправляют данные очень часто (например, каждую секунду), и потеря единичного пакета не критична.